Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Mise en œuvre

Microservice Maintenance

Etudiant :
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

1er avril 2026

Table des matières

1 Objectifs du service Maintenance **2**

2 Modélisation et Persistance **2**
2.1 Base de données dédiée 2
2.2 Entité MaintenanceRecord 2

3 Logique Métier et Alertes **3**
3.1 Gestion des retards (Overdue) 3

4 Intégration et Communication **3**
4.1 Architecture Événementielle 3
4.2 Exposition REST 3
4.3 Exposition via le Gateway GraphQL 4
4.3.1 Agrégation multi-services 4

5 Architecture Événementielle (Saga Pattern) **4**

6 Tests automatisés et couverture de code **4**
6.1 Stratégie et Résultats 4

7 Déploiement Kubernetes et Helm **5**
7.1 Isolation de l'infrastructure (Helm) 5

1. Objectifs du service Maintenance

> **Objectif :** Implémenter le microservice de maintenance permettant la planification des interventions, le suivi de l'historique et la génération d'alertes préventives.

Le service Maintenance est responsable de la santé technique de la flotte. Ses fonctionnalités clés incluent :

- **Planification** : Création et suivi des rendez-vous de maintenance.
- **Historique** : Conservation de toutes les interventions passées pour chaque véhicule.
- **Alertes Préventives** : Détection automatique des retards et des échéances kilométriques.

2. Modélisation et Persistance

Le service respecte le principe d'isolation des données : **un microservice = une base de données**. Il s'appuie sur une instance PostgreSQL dédiée (`postgres-maintenance`), dont le schéma est défini dans `db/03-maintenance.sql`.

2.1. Base de données dédiée

Contrairement aux autres services partageant l'instance principale, la maintenance dispose de sa propre base `maintenance_db` tournant sur un conteneur isolé. Cette architecture garantit qu'une corruption ou une maintenance sur la base des véhicules n'impacte pas le suivi des entretiens.

2.2. Entité MaintenanceRecord

L'entité centrale `MaintenanceRecord` gère les cycles de vie des interventions via plusieurs énumérations (`Type`, `Status`, `Priority`).

```java
@Entity
@Table(name = "maintenance_records")
public class MaintenanceRecord {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private UUID vehicleId; // Lien logique vers le Service Vehicules

    @Enumerated(EnumType.STRING)
    private MaintenanceStatus status = MaintenanceStatus.SCHEDULED;

    private LocalDate scheduledDate;
    private Integer nextServiceKm; // Seuil pour les alertes preventives
}
```

Listing 1 – Entité MaintenanceRecord

3. Logique Métier et Alertes

Le service implémente des mécanismes proactifs pour assurer le suivi de la maintenance.

3.1. Gestion des retards (Overdue)

Une tâche planifiée (`@Scheduled`) s'exécute quotidiennement pour identifier les maintenances prévues dont la date est dépassée.

L'expression Cron utilisée est `"0 0 1 * * ?"`. Elle se décompose comme suit :

- `0` (1er) : 0 seconde.
- `0` (2e) : 0 minute.
- `1` (3e) : 1 heure du matin.
- `*` (4e) : Chaque jour du mois.
- `*` (5e) : Chaque mois.
- `?` (6e) : N'importe quel jour de la semaine.

Ainsi, le système vérifie automatiquement les retards chaque nuit à 01h00.

```
@Scheduled(cron = "0 0 1 * * ?")
@Transactional
public void checkOverdueMaintenance() {
    List<MaintenanceRecord> scheduledRecords = repository.
        findByStatus(MaintenanceStatus.SCHEDULED);
    LocalDate today = LocalDate.now();
    for (MaintenanceRecord record : scheduledRecords) {
        if (record.getScheduledDate().isBefore(today)) {
            record.setStatus(MaintenanceStatus.OVERDUE);
            repository.save(record);
            eventProducer.publishAlert(AlertEvent.overdue(
                record.getVehicleId(), "Maintenance en retard"));
        }
    }
}
```

Listing 2 – Tâche de vérification des retards

4. Intégration et Communication

4.1. Architecture Événementielle

Le service produit des messages sur le topic Kafka `alerts-topic`. Ces événements (`AlertEvent`) sont consommés par le service d'événements pour être notifiés aux gestionnaires.

4.2. Exposition REST

Une API complète a été développée pour permettre au Gateway de gérer les interventions :

Méthode	Endpoint	Action
POST	`/api/maintenance`	Planifier une intervention
GET	`/api/maintenance/vehicle/{id}`	Historique par véhicule
PATCH	`/api/maintenance/{id}/status`	Mettre à jour le statut

4.3. Exposition via le Gateway GraphQL

Le service est pleinement intégré à l'API Gateway via GraphQL.

4.3.1. Agrégation multi-services

L'agrégation permet de croiser les données de trois services distincts :

- **Maintenance → Véhicule** : Récupération des caractéristiques techniques (Service Véhicules).
- **Maintenance → Technicien** : Résolution de l'objet `Driver` lié au `technician_id` (Service Conducteurs).
- **Véhicule → Maintenance** : Affichage de l'historique depuis la fiche véhicule.

```sql
-- Le champ JSONB parts_used en SQL est expose comme [String!]! en GraphQL
CREATE TABLE maintenance_records (
  ...
  parts_used JSONB NOT NULL DEFAULT '[]',
  ...
);
```

Listing 3 – Cohérence des types SQL/GraphQL

5. Architecture Événementielle (Saga Pattern)

Le service participe au cycle de vie global via Kafka :

- **Producers** : Publication sur `maintenance-topic` (`scheduled`, `started`, `completed`, `cancelled`).
- **Consumers** : Écoute du topic `vehicle-events` pour réagir aux mises à jour de kilométrage.

6. Tests automatisés et couverture de code

> **Objectif :** Garantir la robustesse via une couverture minimale de 80%.

6.1. Stratégie et Résultats

- **Tests Unitaires** : JUnit 5 + Mockito pour isoler la logique métier.
- **Tests d'Intégration** : MockMvc + base H2 pour valider l'API et le RBAC.
- **Couverture JaCoCo** : Supérieure à 80% sur l'ensemble du périmètre métier.

7. Déploiement Kubernetes et Helm

7.1. Isolation de l'infrastructure (Helm)

— **Chart fleet-infra** : Alias pour déployer un second StatefulSet (`postgres-maintenance-service`)
— **Initialisation** : ConfigMap `maintenance-db-init` injectant `03-maintenance.sql`.